

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 3**



Build a Scrollable List

Oleh:

Muhammad Naufal Khalish

NIM. 2410817110004

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 3

Laporan Praktikum Pemrograman Mobile Modul 3: Build a Scrollable List ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Naufal Khalish

NIM : 2410817110004

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom.,
M.Kom
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN	ii
DAFTAR ISI	iii
DAFTAR GAMBAR	iv
DAFTAR TABEL	v
SOAL 1	6
A. Source Code	9
B. Output Program	42
C. Pembahasan	44
SOAL 2	56
SOAL 3	57
ADDITIONAL:	58
DAFTAR PUSTAKA	59
TAUTAN GIT	60

DAFTAR GAMBAR

Gambar 1 Contoh UI List.....	7
Gambar 2 Contoh UI Detail	7
Gambar 3 Contoh Penerapan LazyRow/Carousel.....	8
Gambar 4 Tampilan Aplikasi XML	42
Gambar 5 Tampilan Aplikasi XML (Landscape).....	42
Gambar 6 Tampilan Aplikasi Compose	43
Gambar 7 Tampilan Aplikasi Compose	43

DAFTAR TABEL

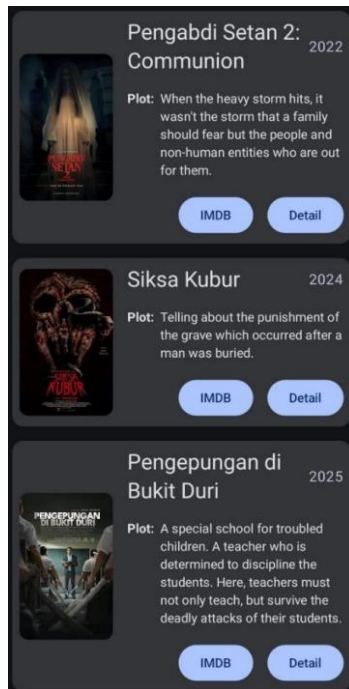
Table 1 Source Code Song.kt XML Soal 1	9
Table 2 Source Code SongData.kt XML Soal 1	9
Table 3 Source Code DetailFragment.kt XML Soal 1	11
Table 4 Source Code HighlightAdapter.kt XML Soal 1	12
Table 5 Source Code HomeFragment.kt XML Soal 1	13
Table 6 Source Code SongListAdapter.kt XML Soal 1	16
Table 7 Source Code SettingsFragment.kt XML Soal 1	17
Table 8 Source Code LanguageSettings.kt XML Soal 1	19
Table 9 Source Code MainActivity.kt XML Soal 1	20
Table 10 Source Code activity_main.xml XML Soal 1	20
Table 11 Source Code fragment_detail.xml XML Soal 1	21
Table 12 Source Code fragment_home.xml XML Soal 1	23
Table 13 Source Code fragment_settings.xml XML Soal 1	24
Table 14 Source Code item_highlight_song.xml XML Soal 1	26
Table 15 Source Code item_song.xml XML Soal 1	26
Table 16 Source Code menu_home.xml XML Soal 1	29
Table 17 Source Code Song.kt Compose Soal 1	30
Table 18 Source Code SongData.kt Compose Soal 1	30
Table 19 Source Code AppNavigation.kt Compose Soal 1	32
Table 20 Source Code DetailScreen.kt Compose Soal 1	32
Table 21 Source Code HomeScreen.kt Compose Soal 1	34
Table 22 Source Code SettingScreen.kt Compose Soal 1	38
Table 23 Source Code Theme.kt Compose Soal 1	40
Table 24 Source Code MainActivity.kt Compose Soal 1	40

SOAL 1

Buatlah sebuah aplikasi Android menggunakan XML dan Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) dan LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut:
 - a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
 - b. Button kedua menggunakan Navigation component untuk membuka laman detail item
5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
8. Aplikasi berbasis XML harus menggunakan ViewBinding
9. Aplikasi Mendukung Multi-language (bahasa indonesia dan inggris)
10. Terdapat 1 halaman/screen untuk melakukan pergantian bahasa
11. Gunakan Icon dari library 'material' (material,material2,material3) untuk membuat tombol yang mengarahkan ke halaman untuk melakukan pergantian bahasa, dan letakkan di bagian header/heading homescreen utama
12. Diatas Scrollable list, terdapat sebuah "Item Highlight" yang menggunakan Carousel atau LazyRow dengan ketentuan:

UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dan setiap baris memiliki 2 teks yang berbeda. Diusahakan agar desain UI item list menyerupai UI berikut:

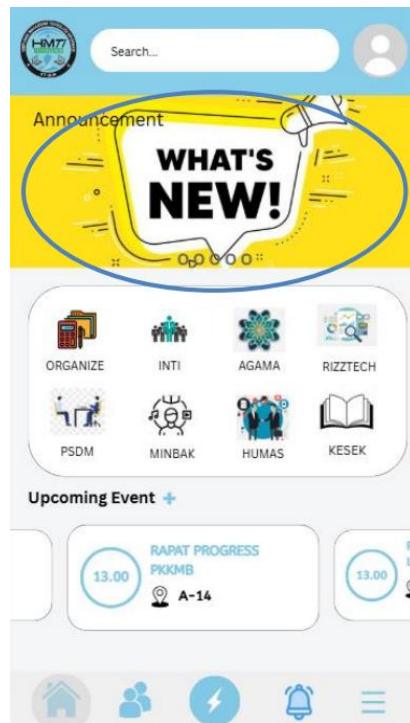


Gambar 1 Contoh UI List

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut:



Gambar 2 Contoh UI Detail



Gambar 3 Contoh Penerapan LazyRow/Carousel

Item yang bergeser bisa secara finite (mentok kiri kanan) atau secara infinite loop (misal dari item terakhir '5' kembali ke item awal '1')

A. Source Code

Source Code XML

Song.kt

Table 1 Source Code Song.kt XML Soal 1

1	package com.example.modul3xml.data
2	
3	data class Song(
4	val id: Int,
5	val title: String,
6	val year: String,
7	val meanId: String,
8	val MeanEn: String,
9	val imageRes: Int,
10	val url: String
11	)

SongData.kt

Table 2 Source Code SongData.kt XML Soal 1

1	package com.example.modul3xml.data
2	
3	import com.example.modul3xml.R
4	
5	object SongData {
6	
7	val SongList = listOf(
8	Song(
9	id = 1,
10	title = "Scott Street",
11	year = "2017",
12	meanId = "Lagu emosional tentang nostalgia dan
	perasaan terasing dari Phoebe Bridgers.",
13	MeanEn = "An emotional song about nostalgia and
	feeling alienated by Phoebe Bridgers.",
14	imageRes = R.drawable.scootstreet,
15	url =
	"https://open.spotify.com/track/21uFPefbgeR3QLVJWATlrr?si=a
	b2c7ad661ad4168"
16	),
17	Song(
18	id = 2,
19	title = "The Night We Met",

```

20         year = "2015",
21         meanId = "Balada melankolis dari Lord Huron
tentang kerinduan untuk kembali ke masa lalu.",
22         MeanEn = "A melancholy ballad by Lord Huron
about longing to return to the past.",
23         imageRes = R.drawable.thenightwemet,
24         url =
"https://open.spotify.com/track/3hRV0jL3vUpRrcy398teAU?si=b
fd097b1bc824f91"
25     ),
26     Song(
27         id = 3,
28         title = "Hello",
29         year = "2015",
30         meanId = "Karya musik folk minimalis dari
Ichiko Aoba dengan petikan gitar yang lembut.",
31         MeanEn = "A folk rock song by Ichiko Aoba with
a soft guitar.",
32         imageRes = R.drawable.hello,
33         url =
"https://open.spotify.com/track/795bHfsDk7EC34IsLqaXfp?si=0
a8eba42ee67448d"
34     ),
35     Song(
36         id = 4,
37         title = "Light Song",
38         year = "2016",
39         meanId = "Karya musik yang menenangkan dengan
harmonis vokal yang indah.",
40         MeanEn = "A soothing musical piece featuring
beautiful vocal harmonies.",
41         imageRes = R.drawable.lightson,
42         url =
"https://open.spotify.com/track/4KXZ4MJKT0RYGhijfQ08Jw?si=9
e08213f078c4a8f"
43     ),
44     Song(
45         id = 5,
46         title = "Childhood",
47         year = "1995",
48         meanId = "Lagu dari daniel.mp3 yang sangat
personal tentang masa lalu saat kecil.",
49         MeanEn = "A personal song about the past when I
was a child.",
50         imageRes = R.drawable.chilhood,
51         url =
"https://open.spotify.com/track/06dDLXf4gPbE02gvH8E6G9?si=1
d289d479e104445"

```

52	)
53	)
54	
55	fun findSongById(songId: Int): Song? {
56	return SongList.find { it.id == songId }
57	}
58	}

DetailFragment.kt

Table 3 Source Code DetailFragment.kt XML Soal 1

1	package com.example.modul3xml.ui.detail
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.ViewGroup
6	import androidx.fragment.app.Fragment
7	import androidx.navigation.fragment.findNavController
8	import com.example.modul3xml.R
9	import com.example.modul3xml.data.SongData
10	import
11	com.example.modul3xml.databinding.FragmentDetailBinding
12	import com.example.modul3xml.util.LanguageSettings
13	class DetailFragment : Fragment() {
14	
15	private var _binding: FragmentDetailBinding? = null
16	private val binding get() = _binding!!
17	
18	override fun onCreateView(
19	inflater: LayoutInflater,
20	container: ViewGroup?,
21	savedInstanceState: Bundle?
22	) = FragmentDetailBinding.inflate(inflater, container,
	false).also {
23	_binding = it
24	}.root
25	
26	override fun onViewCreated(view: android.view.View,
	savedInstanceState: Bundle?) {
27	super.onViewCreated(view, savedInstanceState)
28	
29	setupToolbar()
30	bindSongData()
31	}
32	

33	private fun setupToolbar() {
34	binding.toolbarDetail.setNavigationOnClickListener
	{
35	findNavController().popBackStack()
36	}
37	}
38	
39	private fun bindSongData() {
40	val songId = arguments?.getInt("songId") ?: -1
41	val song = SongData.findSongById(songId)
42	
43	if (song == null) {
44	binding.toolbarDetail.title =
45	getString(R.string.title_detail)
46	return
47	}
48	val isIndonesian =
	LanguageSettings.isIndonesian(requireContext())
49	
50	binding.toolbarDetail.title = song.title
51	binding.imageDetail.setImageResource(song.imageRes)
52	binding.textDetailTitle.text = song.title
53	binding.textDetailYear.text = song.year
54	binding.textDetailMean.text = if (isIndonesian)
	song.meanId else song.MeanEn
55	}
56	
57	override fun onDestroyView() {
58	super.onDestroyView()
59	_binding = null
60	}
61	}

HighlightAdapter.kt

Table 4 Source Code HighlightAdapter.kt XML Soal 1

1	package com.example.modul3xml.ui.home
2	
3	import android.view.LayoutInflater
4	import android.view.ViewGroup
5	import androidx.recyclerview.widget.RecyclerView
6	import com.example.modul3xml.data.Song
7	import
	com.example.modul3xml.databinding.ItemHighlightSongBinding

8	
9	class HighlightAdapter(
10	private val song: List<Song>
11	) :
	RecyclerView.Adapter<HighlightAdapter.HighlightViewHolder>()
	{
12	
13	override fun onCreateViewHolder(parent: ViewGroup,
	viewType: Int): HighlightViewHolder {
14	val binding = ItemHighlightSongBinding.inflate(
15	LayoutInflater.from(parent.context),
16	parent,
17	false
18	)
19	return HighlightViewHolder(binding)
20	}
21	
22	override fun onBindViewHolder(holder:
23	HighlightViewHolder, position: Int) {
	holder.bind(song[position])
24	}
25	
26	override fun getItemCount(): Int = song.size
27	
28	class HighlightViewHolder(private val binding:
	ItemHighlightSongBinding) :
29	RecyclerView.ViewHolder(binding.root) {
30	
31	fun bind(song: Song) {
32	binding.imageHighlight.setImageResource(song.imageRes)
33	}
34	}
35	}

HomeFragment.kt

Table 5 Source Code HomeFragment.kt XML Soal 1

1	package com.example.modul3xml.ui.home
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.os.Bundle
6	import android.view.LayoutInflater
7	import android.view.ViewGroup
8	import androidx.fragment.app.Fragment
9	import androidx.navigation.fragment.findNavController
10	import androidx.recyclerview.widget.LinearLayoutManager

```

11 import com.example.modul3xml.R
12 import com.example.modul3xml.data.SongData
13 import com.example.modul3xml.databinding.FragmentHomeBinding
14 class HomeFragment : Fragment() {
15     private var _binding: FragmentHomeBinding? = null
16     private val binding get() = _binding!!
17
18     override fun onCreateView(
19         inflater: LayoutInflater,
20         container: ViewGroup?,
21         savedInstanceState: Bundle?
22     ): android.view.View {
23         _binding = FragmentHomeBinding.inflate(inflater,
24 container, false)
25         return binding.root
26     }
27
28     override fun onViewCreated(view: android.view.View,
29 savedInstanceState: Bundle?) {
30         super.onViewCreated(view, savedInstanceState)
31
32         setupMenu()
33         setupHighlightList()
34         setupSongList()
35     }
36     private fun setupMenu() {
37         binding.toolbarHome.setOnMenuItemClickListener {
38 item ->
39             when (item.itemId) {
40                 R.id.menu_settings -> {
41 findNavController().navigate(R.id.action_homeFragment_to_set
42 tingsFragment)
43                 true
44             }
45             else -> false
46         }
47     }
48     binding.toolbarHome.title =
49 getString(R.string.app_name)
50 }
51
52     private fun setupHighlightList() {
53         binding.rvHighlight.layoutManager =

```

```

52   LinearLayoutManager(requireContext(),
   LinearLayoutManager.HORIZONTAL, false)
       binding.rvHighlight.adapter =
53   HighlightAdapter(SongData.SongList)
       }
54
55   private fun setupSongList() {
56       binding.rvSong.layoutManager =
57   LinearLayoutManager(requireContext())
       binding.rvSong.adapter = SongListAdapter(
58       songs = SongData.SongList,
59       onBrowserClick = { song ->
60           startActivity(Intent(Intent.ACTION_VIEW,
61   Uri.parse(song.url)))
       },
62       onDetailClick = { song ->
63           val bundle = Bundle().apply {
64               putInt("songId", song.id)
65           }
66
67       findNavController().navigate(R.id.action_homeFragment_to_detailFragment, bundle)
       }
68   )
69   }
70
71   override fun onDestroyView() {
72       super.onDestroyView()
73       _binding = null
74   }
75
76   override fun onResume() {
77       super.onResume()
78
79       _binding?.let { binding ->
80           binding.toolbarHome.title =
81   getString(R.string.app_name)
       }
82   }
83   }
84

```

SongListAdapter.kt

Table 6 Source Code SongListAdapter.kt XML Soal 1

1	package com.example.modul3xml.ui.home
2	
3	import android.view.LayoutInflater
4	import android.view.ViewGroup
5	import androidx.recyclerview.widget.RecyclerView
6	import com.example.modul3xml.data.Song
7	import com.example.modul3xml.databinding.ItemSongBinding
8	import com.example.modul3xml.util.LanguageSettings
9	
10	class SongListAdapter(
11	private val songs: List<Song>,
12	private val onBrowserClick: (Song) -> Unit,
13	private val onDetailClick: (Song) -> Unit
14	) : RecyclerView.Adapter<SongListAdapter.SongViewHolder>()
15	{
16	override fun onCreateViewHolder(parent: ViewGroup,
17	viewType: Int): SongViewHolder {
18	val binding = ItemSongBinding.inflate(
19	LayoutInflater.from(parent.context),
20	parent,
21	false
22	)
23	return SongViewHolder(binding)
24	}
25	override fun onBindViewHolder(holder: SongViewHolder,
26	position: Int) {
27	holder.bind(songs[position], onBrowserClick,
28	onDetailClick)
29	}
30	override fun getItemCount(): Int = songs.size
31	class SongViewHolder(private val binding:
32	ItemSongBinding) :
33	RecyclerView.ViewHolder(binding.root) {
34	fun bind(
35	song: Song,
36	onBrowserClick: (Song) -> Unit,
37	onDetailClick: (Song) -> Unit
38	) {
39	val isIndonesian =
	LanguageSettings.isIndonesian(binding.root.context)

40	
41	binding.apply {
42	imageSong.setImageResource(song.imageRes)
43	textTitle.text = song.title
44	textYear.text = song.year
45	textMean.text = if (isIndonesian)
	song.meanId else song.MeanEn
46	
47	buttonBrowser.setOnClickListener {
	onBrowserClick(song) }
48	buttonDetail.setOnClickListener {
	onDetailClick(song) }
49	}
50	}
51	}
52	}

SettingsFragment.kt

Table 7 Source Code SettingsFragment.kt XML Soal 1

1	package com.example.modul3xml.ui.settings
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.ViewGroup
6	import androidx.fragment.app.Fragment
7	import androidx.navigation.fragment.findNavController
8	import com.example.modul3xml.R
9	import
	com.example.modul3xml.databinding.FragmentSettingsBinding
10	import com.example.modul3xml.util.LanguageSettings
11	
12	class SettingsFragment : Fragment() {
13	
14	private var _binding: FragmentSettingsBinding? = null
15	private val binding get() = _binding!!
16	
17	override fun onCreateView(
18	inflater: LayoutInflater,
19	container: ViewGroup?,
20	savedInstanceState: Bundle?
21	): android.view.View {
22	_binding =
	FragmentSettingsBinding.inflate(inflater, container, false)
23	return binding.root
24	}
25	

```

26     override fun onCreateView(view: android.view.View,
27         savedInstanceState: Bundle?) {
28         super.onCreateView(view, savedInstanceState)
29         setupToolbar()
30         setupLanguageButtons()
31     }
32
33     private fun setupToolbar() {
34         binding.toolbarSettings.title =
getString(R.string.title_settings)
35 binding.toolbarSettings.setNavigationOnClickListener {
36     findNavController().popBackStack()
37 }
38 }
39
40     private fun setupLanguageButtons() {
41         binding.buttonIndonesia.setOnClickListener {
42             LanguageSettings.setLocale(requireActivity(),
43 "id")
44             refreshTexts()
45         }
46         binding.buttonEnglish.setOnClickListener {
47             LanguageSettings.setLocale(requireActivity(),
48 "en")
49             refreshTexts()
50         }
51     }
52     private fun refreshTexts() {
53         binding.toolbarSettings.title =
getString(R.string.title_settings)
54         binding.buttonIndonesia.text =
getString(R.string.btn_indonesia)
55         binding.buttonEnglish.text =
getString(R.string.btn_english)
56     }
57
58     override fun onDestroyView() {
59         super.onDestroyView()
60         _binding = null
61     }
62 }

```

LanguageSettings.kt

Table 8 Source Code LanguageSettings.kt XML Soal 1

1	package com.example.modul3xml.util
2	
3	import android.app.Activity
4	import android.content.Context
5	import androidx.appcompat.app.AppCompatActivity
6	import androidx.core.content.edit
7	import androidx.core.os.LocaleListCompat
8	
9	object LanguageSettings {
10	
11	private const val PREF_NAME = "app_settings"
12	private const val KEY_LANG = "selected_language"
13	
14	fun applySavedLocale(context: Context) {
15	val savedLanguage = getSavedLanguage(context)
16	AppCompatActivity.setApplicationLocales(LocaleListCompat.
17	forLanguageTags(savedLanguage))
18	}
19	fun setLocale(activity: Activity, languageCode:
20	String) {
21	saveLanguage(activity, languageCode)
22	AppCompatActivity.setApplicationLocales(LocaleListCompat.
23	forLanguageTags(languageCode))
24	}
25	fun isIndonesian(context: Context): Boolean {
26	val language =
27	context.resources.configuration.locales[0].language
28	return language == "id" language == "in"
29	}
30	private fun getSavedLanguage(context: Context):
31	String {
32	val prefs =
33	context.getSharedPreferences(PREF_NAME,
34	Context.MODE_PRIVATE)
35	return prefs.getString(KEY_LANG, "id") ?: "id"
36	}
37	
38	private fun saveLanguage(context: Context,
39	languageCode: String) {
40	val prefs =
41	context.getSharedPreferences(PREF_NAME,

	Context.MODE_PRIVATE)
36	prefs.edit {
37	putString(KEY_LANG, languageCode)
38	}
39	}
40	}

MainAcitivity.kt

Table 9 Source Code MainActivity.kt XML Soal 1

1	package com.example.modul3xml
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import
	com.example.modul3xml.databinding.ActivityMainBinding
6	import com.example.modul3xml.util.LanguageSettings
7	
8	class MainActivity : AppCompatActivity() {
9	
10	private lateinit var binding: ActivityMainBinding
11	
12	override fun onCreate(savedInstanceState: Bundle?) {
13	LanguageSettings.applySavedLocale(this)
14	super.onCreate(savedInstanceState)
15	
16	binding =
	ActivityMainBinding.inflate(layoutInflater)
17	setContentView(binding.root)
18	}
19	}

activity_main.xml

Table 10 Source Code activity_main.xml XML Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<FrameLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"

7	android:background="@color/background_black">
8	
9	<androidx.fragment.app.FragmentContainerView
10	android:id="@+id/nav_host_fragment"
11	android:name="androidx.navigation.fragment.NavHostFragment"
12	android:layout_width="match_parent"
13	android:layout_height="match_parent"
14	app:defaultNavHost="true"
15	app:navGraph="@navigation/nav_graph"
16	tools:layout="@layout/fragment_home" />
17	
18	</FrameLayout>

fragment_detail.xml

Table 11 Source Code fragment_detail.xml XML Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	android:fitsSystemWindows="true"
7	android:orientation="vertical"
8	android:background="@color/background_black">
9	
10	<com.google.android.material.appbar.MaterialToolbar
11	android:id="@+id/toolbar_detail"
12	android:layout_width="match_parent"
13	android:layout_height="?attr/actionBarSize"
14	android:paddingTop="8dp"
15	android:background="@color/background_black"
16	app:navigationIcon="@drawable/ic_arrow_back"
17	app:navigationIconTint="@color/white"
18	app:titleTextColor="@color/white" />
19	
20	<ScrollView
21	android:layout_width="match_parent"
22	android:layout_height="match_parent"
23	android:fillViewport="true">
24	
25	<LinearLayout
26	android:layout_width="match_parent"
27	android:layout_height="wrap_content"
28	android:orientation="vertical"

```

29         android:padding="16dp">
30
31     <com.google.android.material.imageview.ShapeableImageView
32         android:id="@+id/image_detail"
33         android:layout_width="match_parent"
34         android:layout_height="260dp"
35         android:contentDescription="@string/cd_song_image"
36         android:clipToOutline="true"
37         android:scaleType="centerCrop"
38         app:shapeAppearanceOverlay="@style/RoundedCornerImage"
39     />
40
41     <LinearLayout
42         android:layout_width="match_parent"
43         android:layout_height="wrap_content"
44         android:layout_marginTop="20dp"
45         android:gravity="center_vertical"
46         android:orientation="horizontal">
47
48         <TextView
49             android:id="@+id/text_detail_title"
50             android:layout_width="0dp"
51             android:layout_height="wrap_content"
52             android:layout_weight="1"
53             android:textColor="@color/white"
54             android:textSize="28sp"
55             android:textStyle="bold" />
56
57         <TextView
58             android:id="@+id/text_detail_year"
59             android:layout_width="wrap_content"
60             android:layout_height="wrap_content"
61             android:textColor="@color/text_gray"
62             android:textSize="22sp" />
63     </LinearLayout>
64
65     <TextView
66         android:layout_width="wrap_content"
67         android:layout_height="wrap_content"
68         android:layout_marginTop="8dp"
69         android:text="@string/title_meaning"
70         android:textColor="@color/white"
71         android:textStyle="bold" />
72
73     <TextView
74         android:id="@+id/text_detail_mean"
75         android:layout_width="match_parent"

```

76	android:layout_height="wrap_content"
77	android:layout_marginTop="8dp"
78	android:textColor="@color/white" />
79	
80	</LinearLayout>
81	
82	</ScrollView>
83	
84	</LinearLayout>

fragment_home.xml

Table 12 Source Code fragment_home.xml XML Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	android:fitsSystemWindows="true"
7	android:orientation="vertical"
8	android:background="@color/background_black">
9	
10	<com.google.android.material.appbar.MaterialToolbar
11	android:id="@+id/toolbar_home"
12	android:layout_width="match_parent"
13	android:layout_height="?attr/actionBarSize"
14	android:background="@color/background_black"
15	app:titleTextColor="@color/white"
16	app:menu="@menu/menu_home" />
17	
18	<androidx.core.widget.NestedScrollView
19	android:layout_width="match_parent"
20	android:layout_height="match_parent"
21	android:fillViewport="true"
22	android:overScrollMode="never">
23	
24	<LinearLayout
25	android:layout_width="match_parent"
26	android:layout_height="wrap_content"
27	android:orientation="vertical">
28	
29	<TextView
30	android:id="@+id/text_highlight"
31	android:layout_width="match_parent"

32	android:layout_height="wrap_content"
33	android:layout_marginStart="16dp"
34	android:layout_marginTop="16dp"
35	android:layout_marginEnd="16dp"
36	android:text="@string/title_highlight"
37	android:textColor="@color/white"
38	android:textSize="16sp"
39	android:textStyle="bold" />
40	
41	<androidx.recyclerview.widget.RecyclerView
42	android:id="@+id/rv_highlight"
43	android:layout_width="match_parent"
44	android:layout_height="120dp"
45	android:layout_marginTop="12dp"
46	android:clipToPadding="false"
47	android:paddingStart="16dp"
48	android:paddingEnd="16dp"
49	android:overScrollMode="never"
50	android:nestedScrollingEnabled="false" />
51	
52	<androidx.recyclerview.widget.RecyclerView
53	android:id="@+id/rv_games"
54	android:layout_width="match_parent"
55	android:layout_height="wrap_content"
56	android:layout_marginTop="32dp"
57	android:clipToPadding="false"
58	android:paddingBottom="8dp"
59	android:overScrollMode="never"
60	android:nestedScrollingEnabled="false" />
61	
62	</LinearLayout>
63	
64	</androidx.core.widget.NestedScrollView>
65	
66	</LinearLayout>
67	

fragment_settings.xml

Table 13 Source Code fragment_settings.xml XML Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:layout_width="match_parent"

```

5      android:layout_height="match_parent"
6      android:fitsSystemWindows="true"
7      android:orientation="vertical"
8      android:background="@color/background_black">
9
10     <com.google.android.material.appbar.MaterialToolbar
11         android:id="@+id/toolbar_settings"
12         android:layout_width="match_parent"
13         android:layout_height="?attr/actionBarSize"
14         android:background="@color/background_black"
15         app:navigationIcon="@drawable/ic_arrow_back"
16         app:navigationIconTint="@color/white"
17         app:titleTextColor="@color/white" />
18
19     <LinearLayout
20         android:layout_width="match_parent"
21         android:layout_height="match_parent"
22         android:orientation="vertical"
23         android:gravity="center"
24         android:paddingTop="16dp"
25         android:paddingHorizontal="48dp">
26
27         <com.google.android.material.button.MaterialButton
28             android:id="@+id/button_indonesia"
29             style="@style/Widget.Material3.Button"
30             android:layout_width="match_parent"
31             android:layout_height="wrap_content"
32             android:text="@string/btn_indonesia"
33             app:backgroundTint="@color/button_blue" />
34
35         <Space
36             android:layout_width="1dp"
37             android:layout_height="16dp" />
38
39         <com.google.android.material.button.MaterialButton
40             android:id="@+id/button_english"
41             style="@style/Widget.Material3.Button"
42             android:layout_width="match_parent"
43             android:layout_height="wrap_content"
44             android:text="@string/btn_english"
45             app:backgroundTint="@color/button_blue" />
46
47     </LinearLayout>
48
49 </LinearLayout>

```

Table 14 Source Code item_highlight_song.xml XML Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<com.google.android.material.card.MaterialCardView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="200dp"
6	android:layout_height="120dp"
7	android:layout_marginEnd="12dp"
8	app:cardBackgroundColor="@color/card_dark"
9	app:cardCornerRadius="12dp"
10	app:cardElevation="2dp"
11	app:strokeWidth="0dp">
12	
13	<ImageView
14	android:id="@+id/image_highlight"
15	android:layout_width="match_parent"
16	android:layout_height="match_parent"
17	android:scaleType="centerCrop"
18	android:contentDescription="@string/cd_song_image"
19	
20	
21	</com.google.android.material.card.MaterialCardView>

Item_song.xml

Table 15 Source Code item_song.xml XML Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<com.google.android.material.card.MaterialCardView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="175dp"
7	android:layout_marginStart="16dp"
8	android:layout_marginEnd="16dp"
9	android:layout_marginBottom="12dp"
10	app:cardBackgroundColor="@color/card_dark"
11	app:cardCornerRadius="16dp"
12	app:cardElevation="2dp"
13	app:strokeWidth="0dp">
14	<LinearLayout
15	android:layout_width="match_parent"
16	android:layout_height="match_parent"
17	android:orientation="horizontal"
18	android:padding="12dp">
19	

```

20     <com.google.android.material.imageview.ShapeableImageView
21         android:id="@+id/image_song"
22     android:layout_width="100dp"
23     android:layout_height="150dp"
24     android:padding="2dp"
25     android:scaleType="centerCrop"
26     app:shapeAppearanceOverlay="@style/RoundedCornerImage" />
27
28     <LinearLayout
29         android:layout_width="0dp"
30         android:layout_height="match_parent"
31         android:layout_weight="1"
32         android:layout_marginStart="12dp"
33         android:orientation="vertical">
34
35         <LinearLayout
36             android:layout_width="match_parent"
37             android:layout_height="wrap_content"
38             android:orientation="horizontal"
39             android:gravity="center_vertical">
40
41             <TextView
42                 android:id="@+id/text_title"
43                 android:layout_width="0dp"
44                 android:layout_height="wrap_content"
45                 android:layout_weight="1"
46                 android:textColor="@color/white"
47                 android:textStyle="bold"
48                 android:maxLines="1"
49                 android:ellipsize="end" />
50
51             <TextView
52                 android:id="@+id/text_year"
53                 android:layout_width="wrap_content"
54                 android:layout_height="wrap_content"
55                 android:textColor="@color/text_gray"
56                 android:textSize="12sp" />
57
58         </LinearLayout>
59
60         <TextView
61             android:id="@+id/text_mean"
62             android:layout_width="match_parent"
63             android:layout_height="wrap_content"
64             android:layout_marginTop="8dp"
65             android:textColor="@color/text_light_gray"
66             android:textSize="12sp"
67             android:lineSpacingExtra="2dp"

```

```

68         android:maxLines="2"
69         android:ellipsize="end" />
70
71     <Space
72         android:layout_width="match_parent"
73         android:layout_height="0dp"
74         android:layout_weight="1" />
75
76     <LinearLayout
77         android:layout_width="match_parent"
78         android:layout_height="wrap_content"
79         android:gravity="bottom|end"
80         android:orientation="horizontal">
81
82         <com.google.android.material.button.MaterialButton
83             android:id="@+id/button_browser"
84             style="@style/Widget.Material3.Button"
85             android:layout_width="wrap_content"
86             android:layout_height="36dp"
87             android:insetTop="0dp"
88             android:insetBottom="0dp"
89             android:minWidth="0dp"
90             android:text="@string/btn_browser"
91             android:textSize="11sp"
92             app:backgroundTint="@color/button_blue"
93         />
94
95         <Space
96             android:layout_width="8dp"
97             android:layout_height="1dp" />
98
99         <com.google.android.material.button.MaterialButton
100             android:id="@+id/button_detail"
101             style="@style/Widget.Material3.Button"
102             android:layout_width="wrap_content"
103             android:layout_height="36dp"
104             android:insetTop="0dp"
105             android:insetBottom="0dp"
106             android:minWidth="0dp"
107             android:text="@string/btn_detail"
108             android:textSize="11sp"
109             app:backgroundTint="@color/button_blue" />
110
111     </LinearLayout>
112
113 </LinearLayout>
114
115 </LinearLayout>

```

115	
116	</com.google.android.material.card.MaterialCardView>

menu_home.xml

Table 16 Source Code menu_home.xml XML Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<menu
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto">
5	<item
6	android:id="@+id/menu_settings"
7	android:icon="@drawable/ic_settings"
8	android:title="@string/title_settings"
9	app:showAsAction="ifRoom" />
10	</menu>

Source Code Compose

Song.kt

Table 17 Source Code Song.kt Compose Soal 1

```
1 package com.example.modul3compose.model
2
3 data class Song(
4     val id: Int,
5     val title: String,
6     val year: String,
7     val meanId: String,
8     val MeanEn: String,
9     val imageRes: Int,
10    val url: String
11 )
```

SongData.kt

Table 18 Source Code SongData.kt Compose Soal 1

```
1 package com.example.modul3compose.data
2
3 import com.example.modul3compose.R
4 import com.example.modul3compose.model.Song
5
6 object SongData {
7     val songLists = listOf(
8         Song(
9             id = 1,
10            title = "Scott Street",
11            year = "2017",
12            meanId = "Lagu emosional tentang nostalgia dan
perasaan terasing dari Phoebe Bridgers.",
13            MeanEn = "An emotional song about nostalgia
and feeling alienated by Phoebe Bridgers.",
14            imageRes = R.drawable.scootstreet,
15            url =
"https://open.spotify.com/track/21uFPefbgeR3QLVJWATlrr?si=
ab2c7ad661ad4168"
16        ),
17        Song(
18            id = 2,
19            title = "The Night We Met",
20            year = "2015",
21            meanId = "Balada melankolis dari Lord Huron
tentang kerinduan untuk kembali ke masa lalu.",
22            MeanEn = "A melancholy ballad by Lord Huron
about longing to return to the past.",
```

23	imageRes = R.drawable.thenightwemet,
24	url = "https://open.spotify.com/track/3hRV0jL3vUpRrcy398teAU?si=bfd097b1bc824f91"
25	), Song(id = 3, title = "Hello", year = "2015", meanId = "Karya musik folk minimalis dari Ichiko Aoba dengan petikan gitar yang lembut.", MeanEn = "A folk rock song by Ichiko Aoba with a soft guitar.", imageRes = R.drawable.hello, url = "https://open.spotify.com/track/795bHfsDk7EC34IsLqaXfp?si=0a8eba42ee67448d"
34	), Song(id = 4, title = "Light Song", year = "2016", meanId = "Karya musik yang menenangkan dengan harmonis vokal yang indah.", MeanEn = "A soothing musical piece featuring beautiful vocal harmonies.", imageRes = R.drawable.lightson, url = "https://open.spotify.com/track/4KXZ4MJKT0RYGhijfQ08Jw?si=9e08213f078c4a8f"
43	), Song(id = 5, title = "Childhood", year = "1995", meanId = "Lagu dari daniel.mp3 yang sangat personal tentang masa lalu saat kecil.", MeanEn = "A personal song about the past when I was a child.", imageRes = R.drawable.childhood, url = "https://open.spotify.com/track/06dDLXf4gPbE02gvH8E6G9?si=1d289d479e104445"
52	)
53	}
54	

AppNavigation.kt

Table 19 Source Code AppNavigation.kt Compose Soal 1

1	package com.example.modul3compose.ui.navigation
2	
3	import androidx.compose.runtime.Composable
4	import androidx.navigation.compose.NavHost
5	import androidx.navigation.compose.composable
6	import androidx.navigation.compose.rememberNavController
7	import com.example.modul3compose.ui.screen.DetailScreen
8	import com.example.modul3compose.ui.screen.HomeScreen
9	import com.example.modul3compose.ui.screen.SettingsScreen
10	
11	@Composable
12	fun AppNavigation() {
13	val navController = rememberNavController()
14	
15	NavHost(navController = navController, startDestination
	= "home") {
16	composable("home") {
17	HomeScreen(navController = navController)
18	}
19	composable("detail/{gameId}") { backStackEntry ->
20	val gameId =
	backStackEntry.arguments?.getString("gameId")?.toIntOrNull()
	DetailScreen(navController = navController,
21	gameId = gameId)
	}
22	composable("settings") {
23	SettingsScreen(navController = navController)
24	}
25	}
26	}
27	

DetailScreen.kt

Table 20 Source Code DetailScreen.kt Compose Soal 1

1	package com.example.modul3compose.ui.screen
2	import android.R
3	import androidx.compose.foundation.*
4	import androidx.compose.foundation.layout.*

```

5 import androidx.compose.foundation.shape.RoundedCornerShape
6 import androidx.compose.material.icons.Icons
7 import
  androidx.compose.material.icons.automirrored.filled.ArrowBack
8
9 import androidx.compose.material3.*
10 import androidx.compose.runtime.Composable
11 import androidx.compose.ui.*
12 import androidx.compose.ui.draw.clip
13 import androidx.compose.ui.graphics.Color
14 import androidx.compose.ui.layout.ContentScale
15 import androidx.compose.ui.platform.LocalConfiguration
16 import androidx.compose.ui.res.painterResource
17 import androidx.compose.ui.res.stringResource
18 import androidx.compose.ui.text.font.FontWeight
19 import androidx.compose.ui.unit.dp
20 import com.example.modul3compose.data.SongData
21
22 @Composable
23 fun DetailScreen(navController:
24 androidx.navigation.NavController, songId: Int?) {
    val song = SongData.songLists.find { it.id == songId }
    val lang =
25 LocalConfiguration.current.locales.get(0).language
26 val isId = lang == "in" || lang == "id"
27
28 Scaffold(
    topBar = {
        Surface(color = Color.Black, shadowElevation =
29 4.dp) {
            Row(Modifier.fillMaxWidth().statusBarsPadding().height(64.dp)
30 ).padding(horizontal = 8.dp), verticalAlignment =
            Alignment.CenterVertically) {
                IconButton(onClick = {
31 navController.popBackStack() }) {
                    Icon(Icons.AutoMirrored.Filled.ArrowBack, null, tint =
32 Color.White)
                }
                Text(song?.title ?: "Detail", style =
34 MaterialTheme.typography.titleLarge, fontWeight =
35 FontWeight.Bold, modifier = Modifier.padding(start = 8.dp))
            }
        }
    }
37
38 ) { p ->
39     song?.let {

```

40	Column(Modifier. <i>fillMaxSize()</i> .padding(p).verticalScroll(rememberScrollState()).padding(16.dp)) {
41	Image(
42	painterResource(it.imageRes), null,
	modifier =
43	Modifier. <i>fillMaxWidth()</i> .height(260.dp).clip(RoundedCornerShape(16.dp)),
44	contentScale = ContentScale.Crop
45	)
	Row(Modifier. <i>fillMaxWidth()</i> .padding(vertical = 20.dp), horizontalArrangement = Arrangement.SpaceBetween)
47	{
	Text(it.title, style =
48	MaterialTheme.typography.headlineMedium, fontWeight =
	FontWeight.Bold, modifier = Modifier.weight(1f))
49	Text(it.year, style =
50	MaterialTheme.typography.titleLarge, color = Color.Gray)
51	}
52	
53	Text(stringResource(com.example.modul3compose.R.string.title_mean), fontWeight = FontWeight.Bold)
54	Text(if (isId) it.meanId else it.MeanEn,
	modifier = Modifier.padding(top = 8.dp))
55	}
56	}
57	}
58	}

HomeScreen.kt

Table 21 Source Code HomeScreen.kt Compose Soal 1

1	package com.example.modul3compose.ui.screen
2	
3	import android.content.Intent
4	import androidx.compose.foundation.Image
5	import androidx.compose.foundation.layout.*
6	import androidx.compose.foundation.lazy.*
7	import androidx.compose.foundation.shape.RoundedCornerShape
8	import androidx.compose.material.icons.Icons
10	import androidx.compose.material.icons.filled.Settings
11	import androidx.compose.material3.*
12	import androidx.compose.runtime.Composable

```

13 import androidx.compose.ui.*
14 import androidx.compose.ui.draw.clip
15 import androidx.compose.ui.graphics.Color
16 import androidx.compose.ui.layout.ContentScale
17 import androidx.compose.ui.platform.*
18 import androidx.compose.ui.res.*
19 import androidx.compose.ui.text.font.FontWeight
20 import androidx.compose.ui.text.style.TextOverflow
21 import androidx.compose.ui.unit.*
22 import androidx.core.net.toUri
23 import com.example.modul3compose.R
24 import com.example.modul3compose.data.SongData
25 import com.example.modul3compose.model.Song

26 @Composable
27 fun HomeScreen(navController:
28 androidx.navigation.NavController) {
29     Scaffold(
30         topBar = {
31             Surface(color = Color.Black, shadowElevation =
32             4.dp) {
33                 Row(
34                     modifier =
35                     Modifier.fillMaxWidth().statusBarsPadding().height(64.dp).padding(horizontal = 16.dp),
36                     verticalAlignment =
37                     Alignment.CenterVertically
38                 ) {
39                     Text(stringResource(R.string.app_name),
40                     style = MaterialTheme.typography.titleLarge, fontWeight =
41                     FontWeight.Bold, modifier = Modifier.weight(1f))
42                     IconButton(onClick = {
43                     navController.navigate("settings") }) {
44                         Icon(Icons.Default.Settings, null,
45                         tint = Color.White)
46                     }
47                 }
48             }
49         }
50     ) { p ->
51         LazyColumn(modifier =
52         Modifier.fillMaxSize().padding(p)) {
53             item {
54                 Text(stringResource(R.string.title_highlight), modifier =
55                 Modifier.padding(16.dp), fontWeight = FontWeight.SemiBold)
56                 LazyRow(contentPadding =
57                 PaddingValues(horizontal = 16.dp), horizontalArrangement =

```

	Arrangement.spacedBy(12.dp)) {
47	items(SongData.songLists) { song ->
48	Card(Modifier.size(200.dp, 120.dp),
	shape = RoundedCornerShape(12.dp)) {
49	Image(painterResource(song.imageRes), null, contentScale =
	ContentScale.Crop, modifier = Modifier.fillMaxSize())
50	}
51	}
52	}
53	Spacer(Modifier.height(24.dp))
54	}
55	items(SongData.songLists) { song ->
56	SongListItem(song) {
	navController.navigate("detail/\${song.id}") }
57	}
58	}
59	}
60	}
61	
62	@Composable
63	fun SongListItem(song: Song, onDetail: () -> Unit) {
64	val ctx = LocalContext.current
65	val config = LocalConfiguration.current
66	
67	val lang = config.locales.get(0).language
68	val isId = lang == "in" lang == "id"
69	
70	Card(
71	modifier = Modifier.padding(horizontal = 16.dp,
	vertical = 8.dp).fillMaxWidth().height(175.dp),
72	shape = RoundedCornerShape(16.dp),
73	colors = CardDefaults.cardColors(containerColor =
	Color(0xFF1A1A1A))
74	) {
75	Row(Modifier.padding(12.dp).fillMaxSize()) {
76	Image(
77	painterResource(song.imageRes), null,
78	modifier = Modifier.size(100.dp,
	150.dp).clip(RoundedCornerShape(12.dp)),
79	contentScale = ContentScale.Crop
80	)
81	Column(Modifier.padding(start =
	12.dp).fillMaxHeight()) {
82	Row(Modifier.fillMaxWidth(),
	horizontalArrangement = Arrangement.SpaceBetween) {
83	Text(song.title, fontWeight =
	FontWeight.Bold, maxLines = 1, overflow =

84	TextOverflow.Ellipsis, modifier = Modifier.weight(1f))
85	Text(song.year, style =
86	MaterialTheme.typography.bodySmall, color = Color.Gray)
87	Text(
88	text = if (isId) song.meanId else
89	song.MeanEn,
90	style =
91	MaterialTheme.typography.bodySmall, color = Color.LightGray,
92	maxLines = 2, overflow =
93	TextOverflow.Ellipsis, modifier = Modifier.padding(top =
94	4.dp), lineHeight = 16.sp
95	)
96	Spacer(Modifier.weight(1f))
97	Row(Modifier.fillMaxWidth(),
98	horizontalArrangement = Arrangement.End) {
99	val blue =
100	ButtonDefaults.buttonColors(containerColor =
101	Color(0xFF2196F3))
102	Button(onClick = {
103	ctx.startActivity(Intent(Intent.ACTION_VIEW,
104	song.url.toUri())) }, colors = blue, modifier =
105	Modifier.height(36.dp)) {
	Text(stringResource(R.string.btn_browser), fontSize = 11.sp)
	Spacer(Modifier.width(8.dp))
	Button(onClick = onDetail, colors =
	blue, modifier = Modifier.height(36.dp)) {
	Text(stringResource(R.string.btn_detail), fontSize = 11.sp)
	}
	}
	}
	}
	}
	}

SettingScreen.kt

Table 22 Source Code SettingScreen.kt Compose Soal 1

1	package com.example.modul3compose.ui.screen
2	
3	import android.app.Activity
4	import android.content.res.Configuration
5	import androidx.activity.compose.LocalActivity
6	import androidx.compose.foundation.layout.*
7	import androidx.compose.material.icons.Icons
8	import androidx.compose.material.icons.automirrored.filled.ArrowBack
9	import androidx.compose.material3.*
10	import androidx.compose.runtime.Composable
11	import androidx.compose.ui.*
12	import androidx.compose.ui.graphics.Color
13	import androidx.compose.ui.res.stringResource
14	import androidx.compose.ui.text.font.FontWeight
15	import androidx.compose.ui.unit.dp
16	import com.example.modul3compose.R
17	import java.util.Locale
18	
19	@Composable
20	fun SettingsScreen(navController: androidx.navigation.NavController) {
21	val act = LocalActivity.current
22	val blue = ButtonDefaults.buttonColors(containerColor = Color(0xFF2196F3))
23	
24	Scaffold(25 topBar = { 26 Surface(color = Color.Black, shadowElevation = 4.dp) {
27	Row(28 modifier = Modifier.fillMaxWidth().statusBarsPadding().height(64.dp).padding(horizontal = 8.dp), 29 verticalAlignment = Alignment.CenterVertically 30) { 31 IconButton(onClick = { navController.popBackStack() }) { 32 Icon(Icons.AutoMirrored.Filled.ArrowBack, null, tint = Color.White) 33 }

34	Text(stringResource(R.string.title_settings), style = MaterialTheme.typography.titleLarge, fontWeight = FontWeight.Bold)
35	}
36	}
37	}
38	) { p ->
39	Column(40 modifier = Modifier.fillMaxSize().padding(p), 41 horizontalAlignment = Alignment.CenterHorizontally, 42 verticalArrangement = Arrangement.Center 43) {
44	LanguageButton(stringResource(R.string.btn_indonesia), blue) { act?.let { updateLang(it, "id") } }
45	Spacer(Modifier.height(16.dp))
46	LanguageButton(stringResource(R.string.btn_english), blue) { act?.let { updateLang(it, "en") } }
47	}
48	}
49	}
50	
51	@Composable
52	fun LanguageButton(text: String, colors: ButtonColors, onClick: () -> Unit) {
53	Button(54 onClick = onClick, 55 modifier = Modifier.fillMaxWidth().padding(horizontal = 48.dp), 56 colors = colors 57) { 58 Text(text) 59 }
60	}
61	
62	@Suppress("DEPRECATION")
63	private fun updateLang(act: Activity, code: String) { 64 val locale = Locale(code) 65 Locale.setDefault(locale) 66 val config = Configuration(act.resources.configuration) 67 config.setLocale(locale) 68 act.resources.updateConfiguration(config, act.resources.displayMetrics) 69 act.recreate() 70 }

Theme.kt

Table 23 Source Code Theme.kt Compose Soal 1

1	package com.example.modul3compose.ui.theme
2	
3	import androidx.compose.material3.MaterialTheme
4	import androidx.compose.material3.darkColorScheme
5	import androidx.compose.runtime.Composable
6	import androidx.compose.ui.graphics.Color
7	
8	private val DarkColorScheme = darkColorScheme(
9	primary = Color.White,
10	background = Color.Black,
11	surface = Color(0xFF121212),
12	onBackground = Color.White,
13	onSurface = Color.White
14	)
15	
16	@Composable
17	fun Modul3Theme(content: @Composable () -> Unit) {
18	MaterialTheme(
19	colorScheme = DarkColorScheme,
20	content = content
21	)
22	}

MainActivity.kt

Table 24 Source Code MainActivity.kt Compose Soal 1

1	package com.example.modul3compose
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.compose.foundation.layout.fillMaxSize
7	import androidx.compose.material3.MaterialTheme
8	import androidx.compose.material3.Surface
9	import androidx.compose.material3.darkColorScheme
10	import androidx.compose.ui.Modifier
11	import androidx.compose.ui.graphics.Color
12	import com.example.modul3compose.ui.navigation.AppNavigation
13	
14	class MainActivity : ComponentActivity() {
15	override fun onCreate(savedInstanceState: Bundle?) {
16	super.onCreate(savedInstanceState)
17	setContent {

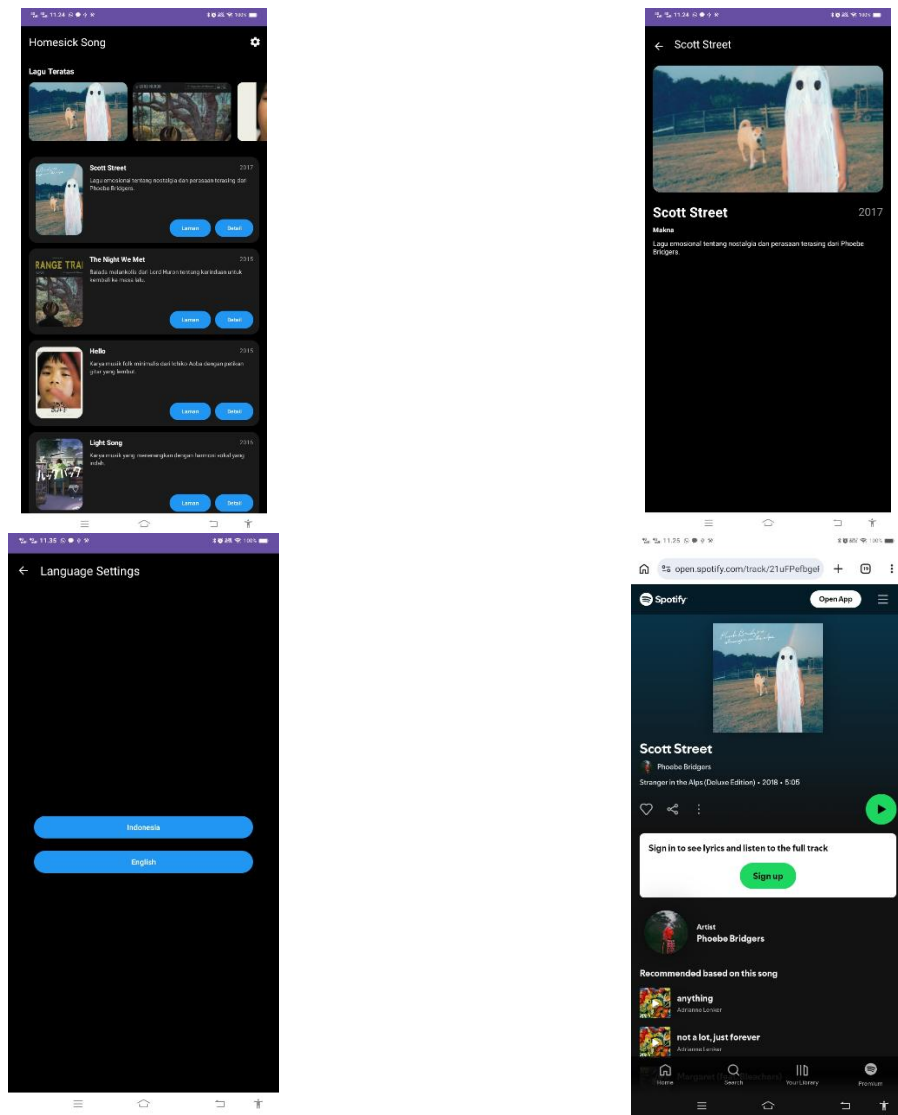
```

18         MaterialTheme(
19             colorScheme = darkColorScheme(
20                 background = Color.Black,
21                 surface = Color(0xFF121212),
22                 onBackground = Color.White,
23                 onSurface = Color.White
24             )
25         ) {
26             Surface(
27                 modifier = Modifier.fillMaxSize(),
28                 color =
MaterialTheme.colorScheme.background
29             ) {
30                 AppNavigation()
31             }
32         }
33     }
34 }
35 }

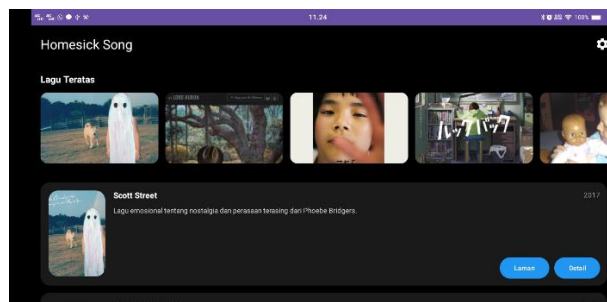
```

B. Output Program

XML

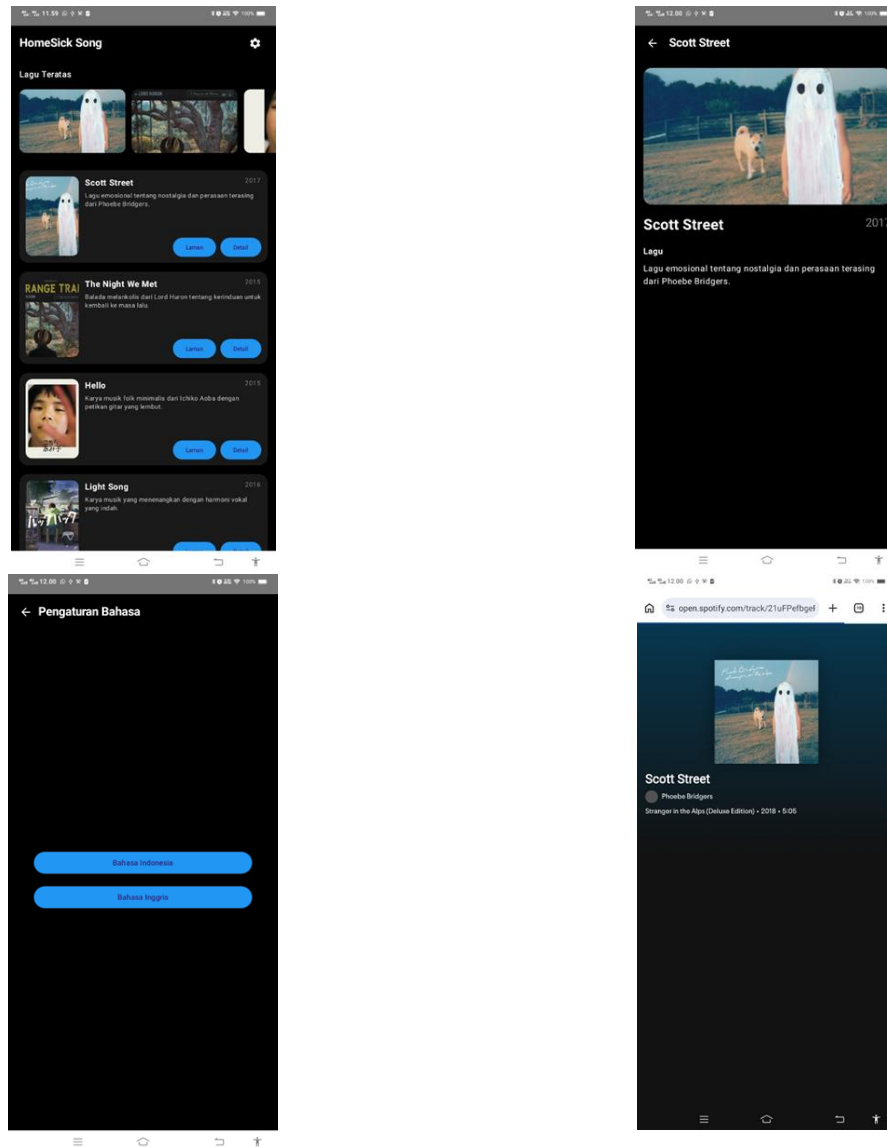


Gambar 4 Tampilan Aplikasi XML

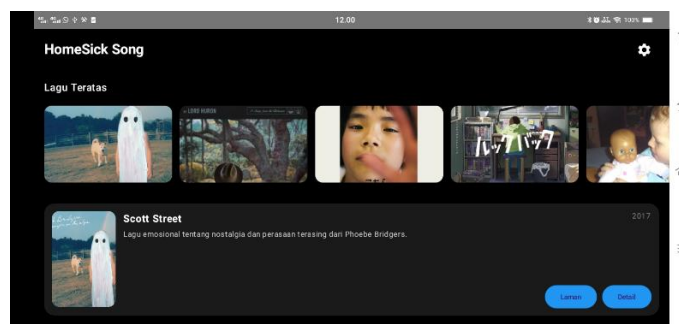


Gambar 5 Tampilan Aplikasi XML (Landscape)

Compose



Gambar 6 Tampilan Aplikasi Compose



Gambar 7 Tampilan Aplikasi Compose

C. Pembahasan

XML

Song.kt

Pada kode Song.kt merupakan data class yang digunakan sebagai model atau wadah data yang tujuannya untuk membungkus informasi pada lagu dalam objek tunggal.

SongData.kt

Kode ini fungsinya sebagai sumber data yang dimana terdapat variabel SongList menyimpan daftar informasi lagu menggunakan struktur data class Song, mencakup detail judul, tahun rilis, deskripsi dalam dua bahasa, gambar dan tautan musik yang akan digunakan pada ui nantinya pada fungsi findSongById digunakan untuk menelusuri lagu ditentukan berdasarkan ID-nya.

DetailFragment.kt

Pada bagian onCreateView, itu merupakan titik awal aplikasi yang dimana saat aplikasi dijalankan untuk mengatur tombol navigasi di toolbar dan mengisi informasi lagu ke layar. Kemudian di bagian setupToolbar itu gunanya saat menekan tombol pada toolbar, maka aplikasi akan menutup screen detail dan mengarahkan ke screen homescreen. Pada bagian bindSongData untuk meletakkan informasi detail lagu dengan mengambil id lagu, dan mencari data nya di daftar lagu yang ada, kemudian ada logika yang mengatur bahasa yang kemudian memasukkan semua informasi seperti judul, tahun, dan gambar ke dalam tampilan secara otomatis. Dan terakhir pada onDestroyView tugasnya membersihkan sisa memori saat screen ditutup.

HighlightAdapter.kt

Bagian onCreateViewHolder mengambil desain dari file XML yaitu item_highlight_song kemudian merakit ke sebuah objek view, kemudian pada bagian onBindViewHolder ini digunakan untuk memasukkan data lagu ke dalam wadah sesuai dengan urutan. Bagian getItemCount digunakan untuk memberitahu seberapa banyak lagu yang ada di dalam daftar. Dan terakhir bagian HighlightViewHolder digunakan untuk menandai elemen gambar lagu

HomeFragment.kt

Pada bagian awal ada inisialisasi Binding dengan dua variabel yaitu _binding dan binding digunakan untuk mengakses elemen desain pada XML lalu onCreateView digunakan untuk membaca desain fragment_home dan disimpan di variabel binding. Lalu bagian onCreateView. Memanggil fungsi fungsi

secara berurutan. bagian `setUpMenu` untuk mengatur bar navigasi di bagian atas atau toolbar. Disini aplikasi menentukan judul halaman dan memberikan logika pada ikon pengaturan bahasa jika ditekan maka akan memindahkan dari screen utama ke screen pengaturan. Bagian `setUpHighlightList`, Bagian ini khusus untuk mengatur daftar lagu yang bisa digeser ke samping (horizontal). Di sini ditentukan bahwa daftar tersebut akan menggunakan data dari `SongData` dan ditampilkan menggunakan `HighlightAdapter` yang sudah dibuat sebelumnya agar gambar lagu muncul berderet ke samping. Bagian `setUpSongList`, Bagian ini mengatur daftar lagu utama yang disusun ke bawah (vertikal). Selain menampilkan data, bagian ini mengatur dua aksi, jika tombol browser diklik maka akan membuka link musik di internet, dan jika item lagu diklik maka akan membawa pengguna masuk ke halaman detail dengan membawa ID lagu tersebut. Bagian `onResume`, fungsinya saat setiap kali pengguna kembali ke home, judul aplikasi pada bar di atas tetap ada dan tidak hilang. Dan terakhir `onDestroyView` seperti pada kode sebelumnya untuk membersihkan memori saat screen ditutup.

SongListAdapter.kt

Pada bagian awal, Adapter ini menerima tiga parameter utama yaitu daftar lagu serta dua fungsi klik `onBrowserClick` dan `onDetailClick` yang berfungsi sebagai instruksi aksi ketika tombol di dalam item ditekan. Bagian `onCreateViewHolder` digunakan untuk menyiapkan desain setiap baris lagu. Di sini, `ItemSongBinding` digunakan untuk membaca file desain `item_song.xml` dan menyimpannya dalam variabel `binding`. Bagian `onBindViewHolder` bertugas untuk menghubungkan data lagu dari urutan tertentu ke dalam tampilan, sedangkan `getItemCount` berfungsi untuk memberitahu sistem jumlah total lagu yang akan ditampilkan dalam daftar. Bagian `SongViewHolder`, khususnya pada fungsi `bind`, merupakan inti dari pengisian data ke elemen visual. Pertama, sistem melakukan pengecekan bahasa melalui `LanguageSettings`. Jika aplikasi menggunakan bahasa Indonesia, maka teks arti lagu akan mengambil data dari `song.meanId`, namun jika tidak, maka akan mengambil dari `song.MeanEn`. Di dalam blok `binding.apply`, semua elemen desain seperti gambar lagu, judul, dan tahun rilis diisi secara otomatis sesuai dengan data yang ada. Terakhir, bagian ini juga mengatur dua aksi klik: jika `buttonBrowser` diklik, maka akan menjalankan fungsi untuk membuka browser, dan jika `buttonDetail` diklik, maka akan menjalankan fungsi untuk menampilkan detail lagu tersebut.

SettingsFragment.kt

Pada bagian awal ada inisialisasi `Binding` dengan dua variabel yaitu `_binding` dan `binding` yang digunakan untuk mengakses elemen desain pada XML. Lalu `onCreateView` digunakan untuk membaca desain `fragment_settings.xml` dan disimpan di variabel `binding` agar komponen UI di dalamnya bisa dikendalikan. Lalu bagian `onViewCreated`, program memanggil fungsi-fungsi secara berurutan untuk menyiapkan tampilan. Bagian `setUpToolbar` berfungsi untuk mengatur bar navigasi di bagian atas. Di sini

aplikasi menentukan judul halaman pengaturan dan memberikan logika pada tombol navigasi kembali; jika ditekan, maka fungsi `popBackStack()` akan dijalankan untuk mengembalikan pengguna ke halaman sebelumnya (Home). Bagian `setUpLanguageButtons` mengatur logika pada dua tombol pilihan bahasa. Jika `buttonIndonesia` ditekan, maka aplikasi akan mengatur bahasa sistem menjadi Indonesia melalui `LanguageSettings`. Begitu juga jika `buttonEnglish` ditekan, maka bahasa akan berubah menjadi Inggris. Setelah bahasa dipilih, fungsi `refreshTexts` akan langsung dipanggil. Bagian `refreshTexts` berfungsi untuk memperbarui teks pada layar secara *real-time*. Fungsi ini mengambil ulang string dari sumber daya aplikasi (`R.string`) untuk judul toolbar dan teks pada tombol. Dan terakhir `onDestroyView`, fungsinya untuk membersihkan memori dengan mengubah variabel `_binding` menjadi null saat screen atau fragment ini ditutup

LanguageSettings.kt

Pada bagian awal, kelas ini didefinisikan sebagai object `LanguageSettings`, yang bertindak sebagai kelas pembantu (*utility*) untuk mengatur lokalisasi atau bahasa di dalam aplikasi. Kelas ini menggunakan `SharedPreferences` dengan identitas `PREF_NAME` dan `KEY_LANG` untuk menyimpan pilihan bahasa pengguna secara permanen di memori perangkat. Bagian `applySavedLocale` berfungsi untuk memuat bahasa yang telah disimpan sebelumnya dan menerapkannya ke seluruh aplikasi. Bagian `setLocale` digunakan saat pengguna menekan tombol ganti bahasa di halaman pengaturan. Di sini, sistem akan melakukan dua hal sekaligus: menyimpan kode bahasa baru (seperti "id" atau "en") ke memori perangkat melalui fungsi `saveLanguage`, dan secara otomatis memperbarui konfigurasi bahasa aplikasi secara *real-time* menggunakan `AppCompatDelegate`. Bagian `isIndonesian` merupakan fungsi pengecekan untuk mengetahui apakah bahasa yang sedang aktif saat ini adalah bahasa Indonesia. Fungsi ini bekerja dengan memeriksa konfigurasi sistem, dan hasilnya (benar atau salah) digunakan oleh bagian lain aplikasi, seperti pada Adapter lagu, untuk menentukan apakah akan menampilkan teks dalam bahasa Indonesia atau Inggris. Terakhir, bagian `getSavedLanguage` dan `saveLanguage` adalah fungsi internal yang bekerja di balik layar. Fungsi-fungsi ini bertugas untuk membaca data dari memori dan menuliskan perubahan data ke dalam pengaturan aplikasi, sehingga pilihan bahasa tetap tersimpan meskipun aplikasi dikeluarkan

MainActivity.kt

Pada bagian awal terdapat inisialisasi variabel binding menggunakan `ActivityMainBinding`. Variabel ini berfungsi sebagai jembatan untuk mengakses seluruh elemen desain yang ada di dalam file `activity_main.xml` secara langsung tanpa perlu menggunakan `findViewById`. Di dalam fungsi `onCreate`, hal pertama yang dilakukan sebelum menampilkan antarmuka adalah memanggil '`LanguageSettings.applySavedLocale(this)`'. Bagian ini berfungsi untuk memastikan bahwa aplikasi langsung menggunakan bahasa yang telah dipilih dan disimpan oleh pengguna (Indonesia atau

Inggris) tepat saat aplikasi baru saja dibuka. Setelah pengaturan bahasa diterapkan, sistem menjalankan perintah `Activity MainBinding.inflate(layoutInflater)` untuk membaca file desain utama. Hasil pembacaan desain tersebut kemudian disimpan ke dalam variabel `binding`. Terakhir, fungsi `setContentView(binding.root)` digunakan untuk menampilkan seluruh komponen visual dari file XML ke layar ponsel pengguna. Karena kelas ini adalah `MainActivity`, maka ia bertugas sebagai kontainer atau wadah utama yang mengatur navigasi dan tempat munculnya `fragment-fragment` lain di dalam aplikasi.

activity_main.xml

Pada bagian awal, desain ini menggunakan `FrameLayout` sebagai kontainer utama yang mencakup seluruh layar aplikasi. Bagian ini memiliki warna latar belakang `@color/background_`. Di dalam `FrameLayout` tersebut terdapat elemen `FragmentContainerView` yang berfungsi sebagai "panggung" atau wadah tempat ditampilkannya berbagai halaman aplikasi secara bergantian. Dengan komponen ini, aplikasi tidak perlu berpindah `Activity`, melainkan cukup menukar konten `Fragment` di dalam satu wadah saja. Atribut `android:name` menetapkan penggunaan `NavHostFragment`, yang merupakan pengatur utama sistem navigasi `Jetpack`. Bagian `app:navGraph` menghubungkan desain ini dengan file alur navigasi (`nav_graph`), yang menentukan urutan halaman dan tujuan perpindahan layar. Bagian `app:defaultNavHost="true"` memastikan bahwa sistem navigasi ini terintegrasi dengan tombol "kembali" (back button) pada ponsel, sehingga ketika pengguna menekan tombol back, aplikasi akan otomatis kembali ke halaman sebelumnya sesuai dengan riwayat navigasi. Terakhir, atribut `tools:layout="@layout/fragment_home"` berfungsi sebagai alat bantu di dalam `Android Studio`. Gunanya agar pada saat proses pengembangan, kita bisa melihat pratinjau atau simulasi tampilan `fragment_home` langsung di dalam editor, meskipun atribut ini tidak akan memberikan pengaruh apa pun saat aplikasi dijalankan di perangkat asli.

fragment_detail.xml

Pada bagian awal, desain ini menggunakan `LinearLayout` sebagai kontainer utama dengan orientasi vertikal dan latar belakang berwarna hitam. Atribut `fitsSystemWindows="true"` ditambahkan agar konten aplikasi tidak tertutup `MaterialToolbar` dengan ID `toolbar_detail` berfungsi sebagai bar navigasi atas. Di sini terdapat sebuah `navigationIcon` berupa panah kembali agar pengguna bisa dengan mudah keluar dari halaman detail. Warna ikon dan judul diatur menjadi putih agar kontras dengan latar belakang hitam yang digunakan. Lalu terdapat `ScrollView` yang membungkus area konten utama. Bagian ini sangat penting agar pengguna bisa menggeser layar ke bawah (scroll) jika informasi lagu yang ditampilkan cukup panjang dan melebihi tinggi layar. Atribut `fillViewport="true"` memastikan konten tetap mengisi ruang layar. Di dalam area konten, terdapat `ShapeableImageView` dengan ID `image_detail` untuk menampilkan gambar lagu dalam ukuran besar. Penggunaan `shapeAppearanceOverlay` memberikan efek sudut

melengkung pada gambar. Bagian `LinearLayout` horizontal di bawah gambar digunakan untuk menata judul lagu dan tahun rilis secara berdampingan. Judul lagu memiliki `layout_weight="1"` agar ia mengambil ruang paling luas di sisi kiri, sedangkan tahun rilis diletakkan di sisi kanan dengan ukuran teks yang sedikit lebih kecil dan warna abu-abu. Terakhir, terdapat label statis bertuliskan "Meaning" yang diambil dari file string sebagai judul informasi, serta sebuah `TextView` dengan ID `text_detail_mean` di bagian paling bawah

fragment_home.xml

Pada bagian awal, desain ini menggunakan `LinearLayout` sebagai kontainer utama dengan orientasi vertikal dan latar belakang berwarna hitam. Atribut `fitsSystemWindows="true"` digunakan agar tata letak aplikasi tidak berbenturan dengan area status bar di bagian atas layar.

Bagian `MaterialToolbar` dengan ID `toolbar_home` berfungsi sebagai bar navigasi utama. Di sini aplikasi menampilkan judul halaman dengan warna teks putih dan menghubungkan file menu (`@menu/menu_home`) yang berisi ikon pengaturan bahasa di pojok kanan atas.

Lalu terdapat `NestedScrollView` yang membungkus seluruh konten halaman. Penggunaan komponen ini memungkinkan pengguna untuk menggeser (scroll) seluruh isi halaman. Atribut `overScrollMode="never"` digunakan untuk menghilangkan efek pantulan saat digeser sampai ujung.

Di dalam konten utama, terdapat sebuah `TextView` yang berfungsi sebagai label judul untuk bagian sorotan (highlight). Teksnya diambil dari sumber daya string (`@string/title_highlight`) dengan gaya huruf tebal dan warna putih.

Bagian `RecyclerView` dengan ID `rv_highlight` digunakan untuk menampilkan daftar lagu pilihan dalam ukuran kecil (tinggi 120dp). Dengan pengaturan yang tepat di bagian kode program, daftar ini akan ditampilkan secara mendatar (horizontal) sehingga gambar-gambar lagu bisa digeser ke samping.

Terakhir, terdapat `RecyclerView` kedua dengan ID `rv_song` yang terletak di bagian bawah. Daftar ini digunakan untuk menampilkan list lagu utama yang disusun memanjang ke bawah (vertikal). Atribut `nestedScrollingEnabled="false"` dipasang pada kedua daftar tersebut agar proses penggeseran layar dikendalikan sepenuhnya oleh `NestedScrollView` induk, sehingga tidak terjadi konflik saat pengguna mengusap layar.

fragemnt_settings.xml

Pada bagian awal, desain ini menggunakan `LinearLayout` sebagai kontainer utama dengan orientasi vertikal dan latar belakang berwarna hitam. Atribut `fitsSystemWindows="true"` digunakan agar tata letak aplikasi tidak berbenturan dengan area status bar di bagian atas layar.

Bagian `MaterialToolbar` dengan ID `toolbar_settings` berfungsi sebagai bar navigasi atas pada halaman pengaturan. Di sini disediakan `navigationIcon` berupa panah kembali dengan warna putih agar pengguna

bisa menavigasi balik ke layar sebelumnya.

Di bawah toolbar, terdapat `LinearLayout` kedua yang berfungsi sebagai wadah utama tombol pilihan bahasa. Wadah ini menggunakan atribut `gravity="center"` agar semua tombol yang ada di dalamnya berada tepat di tengah-tengah layar, serta diberikan jarak (`padding`) agar posisi tombol tidak terlalu menempel ke pinggir layar.

Bagian `MaterialButton` dengan ID `button_indonesia` dan `button_english` digunakan sebagai tombol pilihan bahasa dengan warna biru yang diambil dari `@color/button_blue`

Terakhir, di antara kedua tombol tersebut terdapat elemen `Space` setinggi 16dp. Bagian ini berfungsi untuk memberikan jarak atau ruang kosong yang pas antara tombol bahasa Indonesia dan bahasa Inggris.

item_highlight_song.xml

Pada bagian awal, desain ini menggunakan `MaterialCardView` sebagai kontainer utama yang berfungsi sebagai bingkai untuk setiap item lagu pilihan. Bingkai ini memiliki ukuran lebar 200dp dan tinggi 120dp, serta diberikan jarak di sisi kanan (`layout_marginEnd`) sebesar 12dp.

Bagian kartu ini memiliki sudut melengkung sebesar 12dp dan warna latar belakang gelap (`@color/card_dark`). Terdapat juga sedikit efek bayangan atau `elevation` sebesar 2dp.

Di dalam kartu tersebut, terdapat sebuah `ImageView` dengan ID `image_highlight` yang ukurannya memenuhi seluruh isi kartu. Penggunaan atribut `scaleType="centerCrop"` sangat penting di sini untuk memastikan bahwa gambar lagu yang ditampilkan akan terpotong secara proporsional dan memenuhi seluruh ruang kartu tanpa terlihat gepeng. Terakhir, atribut `contentDescription` ditambahkan untuk mendukung fitur aksesibilitas, memberikan deskripsi pada gambar

item_song.xml

Pada bagian awal, desain ini menggunakan `MaterialCardView` sebagai wadah utama untuk setiap item lagu dalam daftar. Kartu ini memiliki sudut yang melengkung sebesar 16dp dengan warna latar belakang gelap (`@color/card_dark`). Di dalamnya terdapat `LinearLayout` horizontal yang membagi tampilan menjadi dua bagian: gambar di sisi kiri dan informasi teks di sisi kanan. Bagian gambar menggunakan `ShapeableImageView` dengan ID `image_song` yang memiliki ukuran tetap 100x150dp dan sudut melengkung agar serasi dengan bentuk kartu.

Lalu terdapat `LinearLayout` vertikal di sisi kanan yang menampung seluruh informasi lagu. Di bagian paling atas, terdapat susunan horizontal untuk judul lagu (`text_title`) dan tahun rilis (`text_year`). Judul lagu dibuat tebal dan memiliki fitur `ellipsize="end"`, sehingga jika judulnya terlalu panjang, teks akan otomatis terpotong dengan tanda titik-titik. Di bawah judul, terdapat `TextView` dengan ID `text_mean` untuk menampilkan cuplikan makna lagu. Teks ini dibatasi maksimal dua baris dan menggunakan warna abu-abu. Terdapat elemen `Space` dengan berat (`layout_weight="1"`) yang berfungsi sebagai pendorong otomatis

agar bagian tombol selalu berada di posisi paling bawah Terakhir, di sudut kanan bawah terdapat dua `MaterialButton`, yaitu `button_browser` dan `button_detail`. Kedua tombol ini menggunakan gaya Material 3 dengan ukuran yang lebih ringkas (tinggi 36dp) dan warna biru, memberikan opsi kepada pengguna untuk langsung membuka link musik atau masuk ke halaman detail lagu tersebut.

menu_home.xml

Pada bagian awal, kode ini merupakan file konfigurasi Menu yang digunakan untuk menambahkan aksi atau tombol pada bagian Toolbar.

Di dalamnya terdapat sebuah item dengan ID `menu_settings`. Item ini menggunakan ikon yang diambil dari `@drawable/ic_settings`) dan memiliki judul yang merujuk pada string `title_settings`

Bagian `app:showAsAction="ifRoom"` adalah atribut yang memerintahkan sistem agar menampilkan tombol tersebut langsung sebagai ikon di bilah toolbar jika masih ada ruang yang tersedia. Namun, jika layar ponsel terlalu sempit, maka tombol tersebut akan otomatis disembunyikan ke dalam menu titik tiga Terakhir, menu ini nantinya akan dihubungkan dengan kode program (Fragment atau Activity) agar ketika ikon pengaturan tersebut ditekan, aplikasi dapat menjalankan perintah untuk berpindah ke halaman pengaturan.

Compose

Song.kt

Pada bagian awal, kode ini mendefinisikan sebuah data class bernama Song yang berfungsi sebagai model data atau wadah informasi untuk setiap lagu di dalam aplikasi.

Di dalamnya terdapat beberapa variabel utama: id sebagai identitas unik setiap lagu, title untuk menyimpan judul lagu, dan year untuk menyimpan tahun rilisnya. Data-data ini nantinya akan dipanggil oleh komponen UI untuk ditampilkan.

Untuk mendukung fitur multibahasa, kelas ini menyediakan dua variabel teks, yaitu meanId untuk penjelasan makna lagu dalam bahasa Indonesia dan MeanEn untuk penjelasan dalam bahasa Inggris. Pemisahan ini memudahkan aplikasi saat harus memilih teks mana yang akan muncul berdasarkan pengaturan bahasa yang dipilih.

Bagian imageRes menggunakan tipe data integer karena berfungsi untuk menyimpan ID referensi gambar yang ada di folder *drawable* aplikasi. Terakhir, terdapat variabel url yang menyimpan alamat link internet, yang akan digunakan ketika menekan tombol untuk membuka lagu tersebut melalui browser.

SongData.kt

Pada bagian awal, kode ini didefinisikan sebagai object SongData, yang berfungsi sebagai pusat penyimpanan data atau sumber informasi lagu yang akan digunakan di seluruh aplikasi. Dengan menggunakan bentuk object, data ini bersifat statis dan dapat langsung dipanggil tanpa perlu membuat objek baru.

Di dalamnya terdapat sebuah variabel bernama songLists yang berisi sebuah daftar (listOf) dari beberapa objek Song. Daftar ini menjadi kumpulan data utama yang akan ditampilkan pada layar aplikasi, baik dalam bentuk daftar vertikal maupun sorotan (*highlight*).

Setiap lagu di dalam daftar ini telah diisi dengan id, title (judul), dan year (tahun rilis).

Untuk mendukung fitur multibahasa, setiap entri lagu menyertakan penjelasan dalam dua versi, yaitu meanId untuk bahasa Indonesia dan MeanEn untuk bahasa Inggris. Selain itu, bagian imageRes menghubungkan lagu dengan file gambar yang ada di folder *drawable* melalui referensi R.drawable.

Terakhir, setiap lagu juga dilengkapi dengan url yang merujuk pada link streaming musik. Data inilah yang nantinya dikirim ke komponen UI (Compose) untuk diproses, sehingga dapat melihat gambar, membaca makna lagu, hingga membuka link tersebut.

AppNavigation.kt

Pada bagian awal, fungsi Composable AppNavigation ini bertugas sebagai pusat kendali navigasi atau perpindahan layar di dalam aplikasi. Di sini dibuat variabel navController menggunakan rememberNavController() yang berfungsi untuk melacak riwayat kunjungan layar dan mengelola perintah perpindahan antar halaman.

Bagian NavHost digunakan sebagai wadah utama yang mengatur alur navigasi dengan startDestination = "home". Ini artinya, saat aplikasi pertama kali dibuka, sistem akan secara otomatis mengarahkan pengguna untuk melihat layar utama terlebih dahulu.

Di dalam NavHost, terdapat beberapa rute yang didefinisikan menggunakan fungsi composable. Rute pertama adalah "home" yang akan menampilkan HomeScreen. Di sini, variabel navController ikut dikirimkan agar halaman utama tersebut memiliki kemampuan untuk memerintahkan perpindahan ke layar lain, seperti ke halaman detail atau pengaturan.

Bagian rute kedua adalah "detail/{songId}" yang digunakan untuk menuju halaman detail secara dinamis. Rute ini membawa parameter berupa ID lagu yang dipilih pengguna. Di dalamnya, sistem akan mengambil nilai songId dari backStackEntry, mengubahnya menjadi tipe data angka (Integer), lalu mengirimkan ID tersebut ke DetailScreen agar data lagu yang ditampilkan tepat sesuai dengan yang diklik.

Terakhir, terdapat rute "settings" yang akan menampilkan SettingsScreen. Sama seperti rute lainnya, halaman ini juga menerima navController agar pengguna bisa melakukan aksi navigasi, seperti menekan tombol kembali untuk menuju ke layar sebelumnya setelah selesai mengubah pengaturan bahasa.

DetailScreen.kt

Pada bagian awal, fungsi Composable DetailScreen ini menerima dua parameter penting, yaitu navController untuk mengelola navigasi dan songId untuk menentukan lagu mana yang harus ditampilkan. Sistem kemudian mencari data lagu yang cocok dari SongData serta melakukan pengecekan bahasa sistem melalui LocalConfiguration untuk menentukan apakah teks yang muncul nantinya menggunakan bahasa Indonesia atau Inggris.

Bagian Scaffold digunakan sebagai kerangka utama layar yang menyediakan struktur dasar. Di dalam atribut topBar, terdapat Surface berwarna hitam yang berisi sebuah IconButton dengan ikon panah kembali; jika ditekan, fungsi popBackStack() akan dijalankan untuk mengembalikan pengguna ke halaman sebelumnya. Di samping tombol tersebut, judul lagu ditampilkan dengan gaya huruf tebal.

Bagian konten utama dibungkus menggunakan Column yang dilengkapi dengan fungsi verticalScroll. Hal ini bertujuan agar seluruh informasi lagu dapat digeser ke bawah (scroll) dengan lancar jika isinya melebihi tinggi layar ponsel. Seluruh konten diberikan jarak (padding) agar tidak terlalu menempel ke pinggir layar.

Di dalam kolom tersebut, komponen Image diletakkan paling atas untuk menampilkan poster lagu. Gambar ini memiliki tinggi 260dp dan menggunakan RoundedCornerShape(16.dp) untuk memberikan efek sudut melengkung yang estetik, serta ContentScale.Crop agar gambar memenuhi ruang yang tersedia tanpa merusak proporsinya.

Bagian informasi teks diatur menggunakan Row untuk menyusun judul lagu dan tahun rilis secara berdampingan. Judul lagu diberikan bobot (weight) agar mengambil ruang lebih luas, sementara tahun rilis diletakkan di sisi kanan dengan warna abu-abu.

Terakhir, terdapat bagian penjelasan makna lagu. Baris kode ini akan memilih antara menampilkan variabel `it.meanId` atau `it.MeanEn` berdasarkan hasil pengecekan bahasa di awal tadi.

HomeScreen.kt

Pada bagian awal, fungsi Composable HomeScreen menggunakan Scaffold sebagai kerangka utama layar. Di bagian topBar, terdapat Surface berwarna hitam yang menampilkan nama aplikasi di sisi kiri dan sebuah IconButton dengan ikon pengaturan di sisi kanan. Jika ikon tersebut ditekan, maka aplikasi akan menjalankan fungsi navigasi untuk berpindah ke rute "settings".

Bagian konten utama diatur menggunakan LazyColumn agar daftar konten dapat discroll secara vertikal. Di bagian paling atas daftar, terdapat teks judul untuk bagian Highlight yang diikuti oleh sebuah LazyRow. Bagian ini memungkinkan pengguna untuk menscroll daftar gambar lagu pilihan ke arah samping atau horizontal dengan tampilan kartu yang memiliki sudut melengkung.

Di bawah bagian sorotan tersebut, terdapat daftar lagu utama yang disusun menggunakan fungsi items. Setiap data lagu yang diambil dari SongData akan diproses dan ditampilkan satu per satu secara berderet ke bawah melalui komponen SongListItem.

Komponen SongListItem sendiri bertugas untuk mendefinisikan tampilan setiap baris lagu. Di sini, setiap lagu dibungkus di dalam sebuah Card berwarna gelap dengan sudut melengkung 16dp. Di dalamnya, sebuah Row digunakan untuk menyusun gambar lagu di sisi kiri dan kolom informasi di sisi kanan secara berdampingan.

Untuk bagian teks informasi, sistem secara otomatis mendeteksi bahasa ponsel melalui variabel `isId`. Jika bahasa Indonesia, maka aplikasi menampilkan `song.meanId`, namun jika bahasa lain, maka yang muncul adalah `song.MeanEn`. Teks judul dan makna lagu juga dilengkapi dengan fitur Ellipsis, sehingga jika teksnya terlalu panjang, ia akan otomatis terpotong dengan tanda titik-titik agar tetap rapi.

Terakhir, di sudut kanan bawah setiap kartu lagu terdapat dua tombol berwarna biru. Tombol Browser menggunakan Intent untuk membuka alamat URL lagu tersebut secara langsung di aplikasi browser atau Spotify, sedangkan tombol Detail akan menjalankan perintah navigasi untuk membawa pengguna masuk ke halaman DetailScreen dengan membawa ID lagu yang teekait.

SettingsScreen.kt

Pada bagian awal, fungsi Composable SettingsScreen ini menginisialisasi variabel `act` untuk mendapatkan referensi Activity yang sedang berjalan serta menentukan skema warna biru yang akan digunakan secara seragam pada tombol-tombol di halaman ini.

Bagian Scaffold digunakan sebagai kerangka utama layar. Di dalam atribut `topBar`, terdapat sebuah Surface berwarna hitam yang menampung IconButton dengan ikon panah kembali. Jika tombol ini ditekan, fungsi `navController.popBackStack()` akan dijalankan untuk membawa pengguna kembali ke layar sebelumnya. Judul halaman diambil secara dinamis dari sumber daya string agar mendukung multibahasa.

Bagian konten utama diatur menggunakan Column yang memiliki pengaturan `Arrangement.Center` dan `Alignment.CenterHorizontally`. Hal ini bertujuan agar tombol-tombol pilihan bahasa muncul tepat di tengah-tengah layar ponsel. Di dalamnya terdapat dua pemanggilan fungsi `LanguageButton` yang dipisahkan oleh sebuah Spacer setinggi 16dp sebagai ruang kosong antar tombol.

Fungsi `LanguageButton` sendiri adalah sebuah komponen untuk membuat tombol dengan desain yang konsisten. Tombol ini diatur agar memenuhi lebar layar dengan jarak (`padding`) horizontal sebesar 48dp. Saat tombol ditekan, ia akan mengirimkan instruksi ke fungsi `updateLang` dengan membawa kode bahasa yang sesuai, yaitu "id" untuk Indonesia atau "en" untuk Inggris.

Terakhir, terdapat fungsi `updateLang` yang menangani logika teknis penggantian bahasa. Fungsi ini bekerja dengan cara membuat objek `Locale` baru, memperbarui konfigurasi sumber daya (*resources*) pada sistem Android, dan yang paling penting adalah menjalankan perintah `act.recreate()`. Perintah ini akan memulai ulang aktivitas agar seluruh tampilan aplikasi disegarkan secara otomatis menggunakan bahasa yang baru saja dipilih oleh pengguna.

Theme.kt

Pada bagian awal, kode ini mendefinisikan sebuah skema warna yang disimpan dalam variabel `DarkColorScheme`. Di sini, aplikasi dikonfigurasi untuk menggunakan tema gelap (*Dark Mode*) dengan menetapkan warna hitam (`Color.Black`) sebagai latar belakang utama dan warna putih sebagai warna primer untuk elemen-elemen penting.

Di dalam skema tersebut, terdapat pengaturan warna surface dengan kode hex `0xFF121212` yang digunakan sebagai warna latar belakang untuk komponen seperti kartu (*Card*) atau bilah atas (*Toolbar*). Selain itu, warna `onBackground` dan `onSurface` diatur menjadi putih agar semua teks yang muncul di atas latar belakang gelap tersebut tetap kontras dan mudah dibaca.

Lalu terdapat fungsi Composable utama bernama `Modul3Theme`. Fungsi ini bertindak sebagai pembungkus (*wrapper*) atau penyedia tema bagi seluruh konten aplikasi. Dengan menggunakan

MaterialTheme, semua komponen Jetpack Compose yang berada di dalam lingkup fungsi ini akan secara otomatis mengikuti aturan warna yang telah didefinisikan dalam DarkColorScheme.

Terakhir, parameter content memungkinkan fungsi ini menerima blok kode UI lainnya. Hal ini memastikan bahwa seluruh layar aplikasi, mulai dari halaman utama hingga halaman pengaturan, memiliki visual yang konsisten.

MainActivity.kt

Pada bagian awal, kelas MainActivity ini merupakan titik masuk utama aplikasi yang mewarisi sifat dari ComponentActivity. Di dalam fungsi onCreate, sistem akan menyiapkan segala kebutuhan aplikasi tepat saat aplikasi mulai dijalankan di ponsel pengguna.

Bagian setContent digunakan untuk mendefinisikan seluruh antarmuka pengguna menggunakan Jetpack Compose. Di sini, aplikasi tidak lagi menggunakan file XML, melainkan langsung menyusun komponen UI menggunakan kode Kotlin.

Terdapat pengaturan MaterialTheme yang secara khusus mendefinisikan skema warna aplikasi melalui darkColorScheme. Bagian ini memastikan aplikasi tampil dengan tema gelap, di mana latar belakang diatur menjadi hitam, permukaan komponen diatur menjadi abu-abu sangat gelap, serta semua teks yang muncul di atasnya diatur berwarna putih agar tetap kontras dan mudah dibaca.

Di dalam tema tersebut, terdapat komponen Surface yang berfungsi sebagai wadah atau "lantai" paling dasar dari aplikasi. Dengan menggunakan atribut Modifier.fillMaxSize(), Surface ini akan membentang luas hingga memenuhi seluruh ukuran layar ponsel pengguna dan menerapkan warna latar belakang sesuai tema yang telah dibuat.

Terakhir, di dalam wadah Surface tersebut, aplikasi memanggil fungsi AppNavigation(). Bagian inilah yang menjadi perintah penutup untuk memulai sistem navigasi, sehingga aplikasi dapat menampilkan layar pertama (Home) dan mengelola proses perpindahan antar halaman sesuai dengan rute yang telah ditentukan sebelumnya.

SOAL 2

Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

RecyclerView merupakan standar industri yang sudah ada selama satu dekade lebih, jadi pada waktu itu , Google dan komunitas sudah meningkatkan performanya sampai mumpuni. RecyclerView lebih kompatibel di perangkat spesifikasi rendah atau istilahnya ponsel kentang dalam konteks scrolling, dibandingkan dengan LazyColumn yang merupakan teknologi yang lebih baru namun lebih berat bagi prosessor karea sistem Jetpack Compose harus terus render ulang secara real-time, yang ini berpengaruh pada performa perangkat seperti baterai lebih boros dan suhu ponsel yang cepat panas.

Berdasarkan hasil penelitian yang diterbitkan dalam Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer (2026), RecyclerView terbukti jauh lebih efisien dibandingkan LazyColumn dalam hal penggunaan sumber daya, di mana penggunaan CPU rata-rata saat *scrolling* hanya sebesar ~25% berbanding ~45% pada LazyColumn. Selain itu, RecyclerView memiliki konsumsi memori yang lebih rendah, yaitu sekitar ~143 MB dibandingkan LazyColumn yang membutuhkan ~156 MB. Dalam hal stabilitas tampilan, RecyclerView mampu memberikan pengalaman pengguna yang jauh lebih mulus dengan durasi frame rata-rata 4 ms tanpa terjadi *jank*, sementara LazyColumn mencapai 29 ms yang berisiko menyebabkan patah-patah pada animasi[1] .

Kemudian karena RecyclerView lebih mendekati pendekatan XML yang imperatif maka dalam hal detail fitur contohnya dekorasi list lebih spesifik dibandingkan LazyColumn yang menerapkan pendekatan deklaratif. Lalu karena sudah menjadi standar industri maka jika ada perusahaan yang memiliki aplikasi yang besar yang menerapkan RecyclerView karena dipercaya teknologi yang sudah matang dan stabil lalu mengubah semua itu ke LazyColumn yang perlu maintenance dan ujungnya membutuhkan cost yang besar dan ini hanya mempertaruhkan resiko resiko pada masalah kedepannya dan ini didukung juga dengan dokumentasi serta komunitas ini dibuktikan dengan forum pada StackOverflow sudah Support lebih dari satu dekade.

SOAL 3

Pada desain arsitektur kotlin, terdapat beberapa arsitektur dengan salah satunya adalah CLEAN ARCHITECTURE, menurut pendapat kalian, mengapa kita harus menerapkan arsitektur ini ketika membuat sebuah project?

Penerapan clean architecture dalam proyek kotlin memberikan pemisahan tanggung jawab yang jelas atau sering disebut sebagai separation of concerns. Tujuannya untuk mengeaskan Dependency Rule, yang dimana ketergantungan antar komponen hanya boleh mengarah ke Domain Layer. Dalam manajemen proyek , penerapan Clean Architecture sebagai strategi mitigasi risiko. Selain itu, clean architecture jika terjadi perubahan kebutuhan bisnis memberikan fleksibilitas tinggi, karena logika inti terisolasi dari detail teknis, estimasi waktu untuk pembaruan fitur atau penggantian penyedia layanan pihak ketiga agar meminimalkan risiko karena merusak modul lain atau efek domino. Clean architecture sebagai fondasi dalam hal skalabilitas, clean architecture memfasilitasi Modularisasi, setiap lapisan seperti Data, Domain , Presentation terpisah secara logis. Kemudian menghindari jebakan "kode spaghetti" yang sering kali menjadi hambatan utama dalam pengembangan perangkat lunak. Dengan membagi kode ke dalam lapisan-lapisan yang memiliki tanggung jawab spesifik sehingga jika terjadi bug lebih mudah diperbaiki tanpa merusak atau merambat ke lain. Clean Architecture berperan penting dalam memperkuat kolaborasi tim melalui standarisasi dan fleksibilitas pengembangan. Dengan adanya pedoman yang seragam, tim dapat bekerja secara paralel pada modul yang berbeda tanpa risiko saling tumpang tindih atau mengganggu fungsionalitas satu sama lain. Ini memberikan keuntungan ekonomi yang signifikan melalui pengurangan biaya pengembangan dan pemeliharaan. Kode yang bersih dan mudah dipahami secara otomatis meminimalkan risiko kesalahan manusia serta mengurangi waktu yang terbuang untuk memperbaiki masalah teknis yang kompleks.

ADDITIONAL:

Penerapan dan pembahasan clean code

Penerapan clean code pada praktikum ini melalui pemisahan tanggung jawab yang terstruktur dengan baik antara logika data, pengaturan tampilan, dan manajemen navigasi. Dengan memisahkan model data ke dalam kelas Song dan penyimpanan data statis pada objek SongData, kode menjadi lebih mudah dikelola karena setiap bagian memiliki tugas tunggal yang spesifik. Penggunaan penamaan variabel dan fungsi yang deskriptif seperti `onBrowserClick` atau `SongListItem` juga memegang peranan penting dalam meningkatkan keterbacaan kode, sehingga maksud dari setiap baris perintah dapat dipahami secara instan tanpa memerlukan banyak komentar tambahan.

Pada implementasi berbasis XML, penggunaan `ViewBinding` merupakan langkah untuk menghindari kesalahan umum seperti `NullPointerException` yang sering terjadi pada penggunaan `findViewById` secara manual. Selain itu, penggunaan kelas pembantu `LanguageSettings` menunjukkan penerapan modularitas yang baik, di mana logika untuk mengubah bahasa sistem diisolasi dari kelas tampilan utama. Hal ini memastikan bahwa perubahan pada sistem pengaturan tidak akan mengganggu performa halaman beranda maupun halaman detail, sehingga struktur aplikasi tetap stabil dan terorganisir.

Sementara itu, pada versi Jetpack Compose, prinsip clean code diterapkan melalui pembuatan komponen UI yang bersifat reusable atau dapat digunakan kembali, seperti pada fungsi `LanguageButton` dan `SongListItem`. Dengan membungkus elemen UI ke dalam fungsi-fungsi kecil, kode menjadi tidak redundan dan mempermudah proses pemeliharaan jika terjadi perubahan desain di masa depan. Pengaturan tema yang terpusat pada `Modul3Theme` juga memastikan konsistensi visual di seluruh aplikasi, yang merupakan salah satu standar penting dalam penulisan kode yang profesional dan efisien.

Selain itu menerapkan prinsip lokalisasi yang bersih dengan menyimpan teks pada file string dan menyediakan variabel khusus untuk bahasa Indonesia maupun Inggris di dalam model data. Pendekatan ini memungkinkan aplikasi untuk beradaptasi dengan preferensi pengguna tanpa harus menulis ulang logika tampilan berkali-kali.

DAFTAR PUSTAKA

- [1] J. C. Buana, A. P. Kharisma, and B. Priyambadha, “Analisis Perbandingan Performa RecyclerView dan LazyColumn dalam Menampilkan Koleksi Data pada Aplikasi Android,” vol. 10, no. 4, pp. 1–6, 2026.

TAUTAN GIT

Berikut adalah tautan untuk repositori GitHub yang telah dibuat

[Link Github](#)